



From 2.4 Million Sentences to 5 Suggestions

Building a Scalable Autocomplete Search System

2.39M

Sentences Loaded

5

Top Suggestions

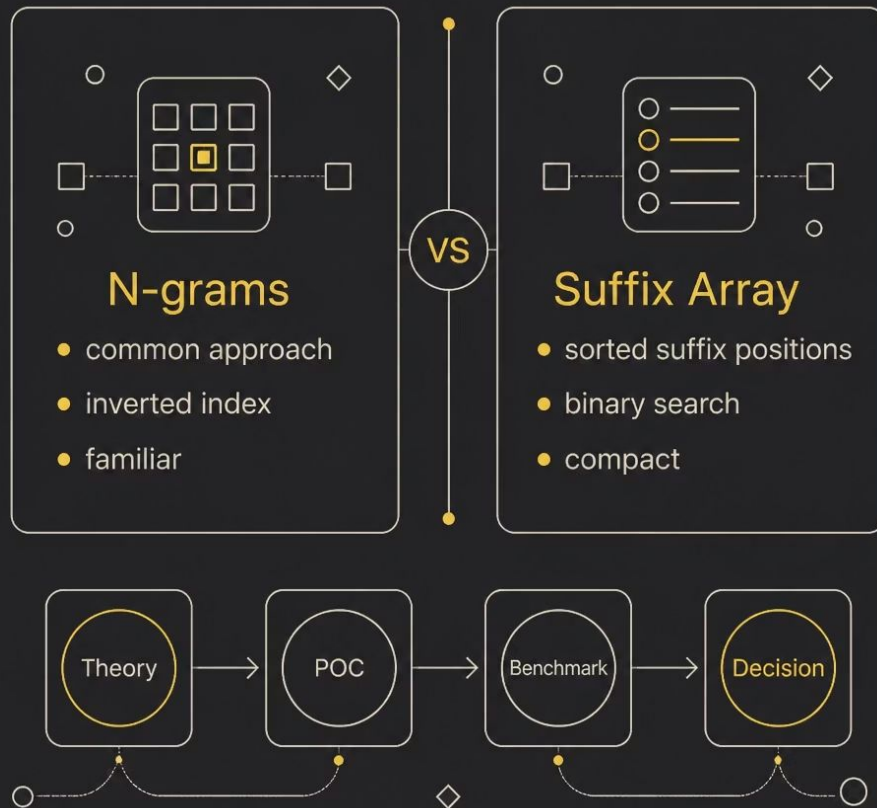
TEAM PROJECT

END-TO-END SYSTEM



Baseline first. A correct end-to-end system became our correctness reference for every later optimization.

Choosing the Search Strategy

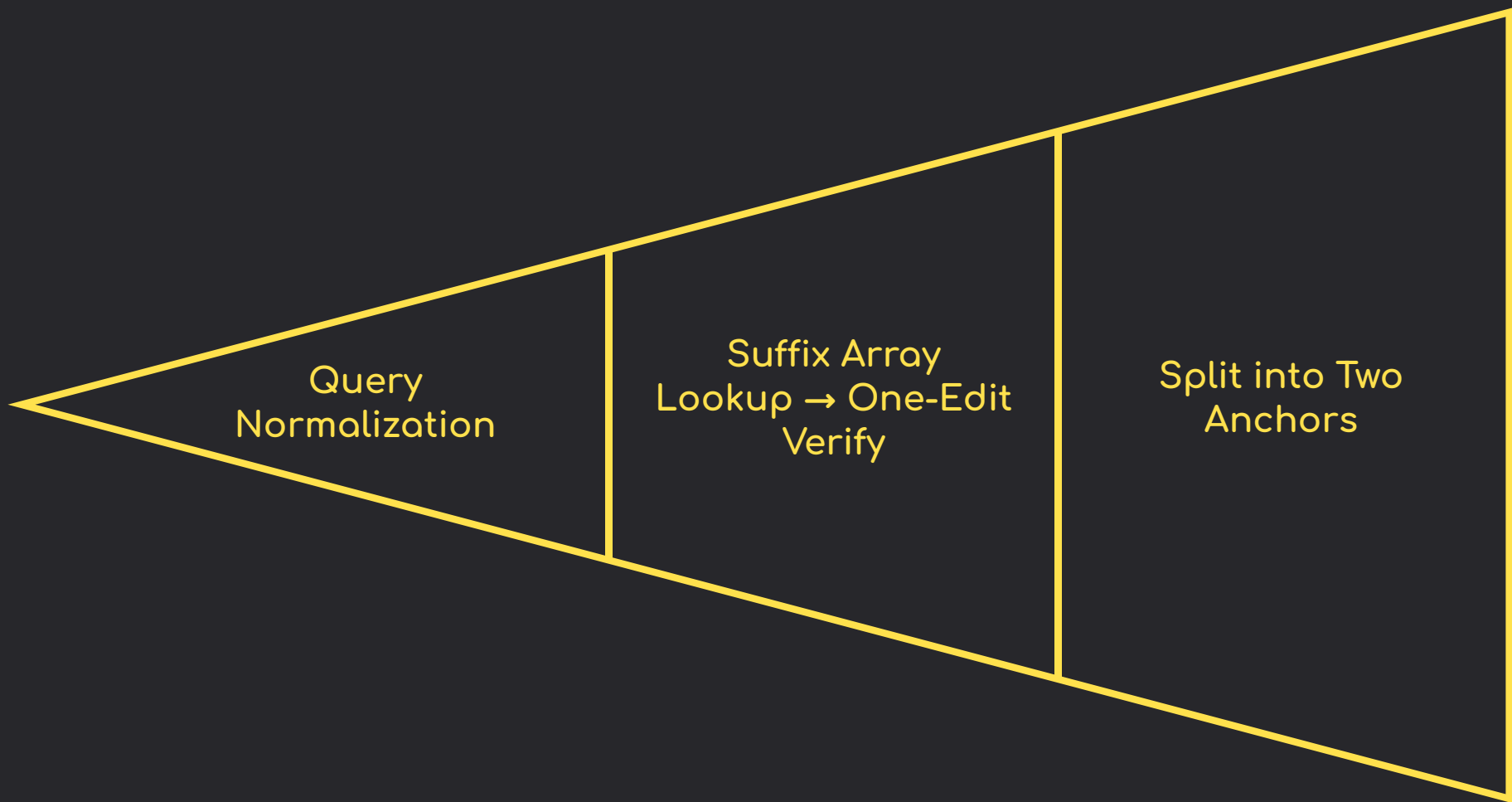


The Decision Process

- Initial direction: N-grams
- Research surfaced Suffix Arrays
- Built a POC — then benchmarked

"Don't decide only theoretically — benchmark it."

From a Query to a Small Candidate Set



One edit cannot corrupt both anchors — at least one stays exact.

Example: machine lerning

machine

Exact if error is in
second anchor

lerning

Exact if error is in
first anchor

Three Roles

Suffix Array

Who *might* match?

One-Edit Verifier

Who *actually* matches?

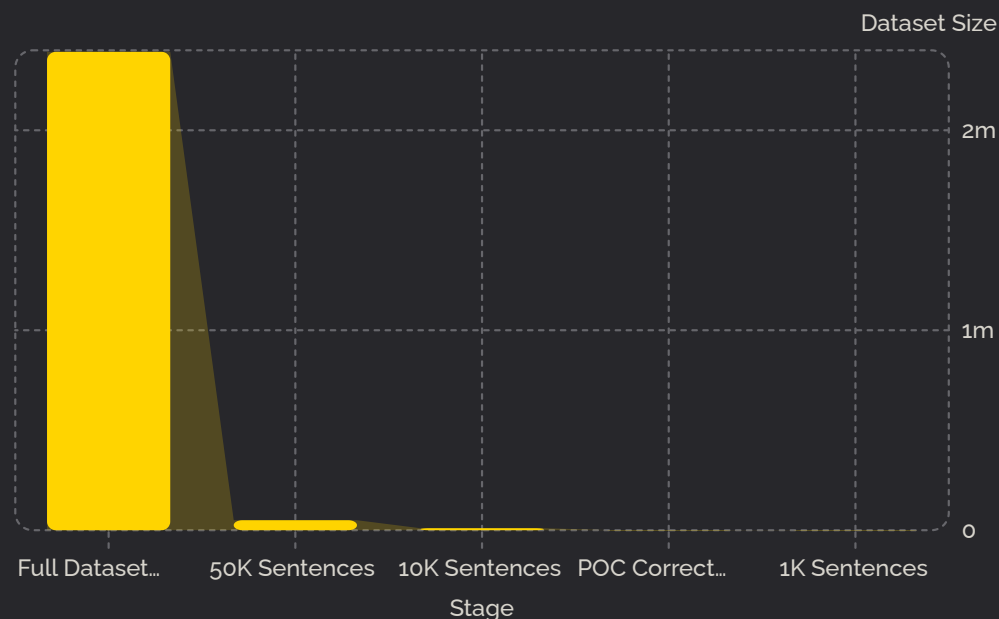
Scoring

Which are *best*?

The Algorithm Worked — Python Didn't Scale

0.04%

Total conversion rate



The Scaling Wall

Algorithm correct. **Python representation became the bottleneck.**

Python · 50K

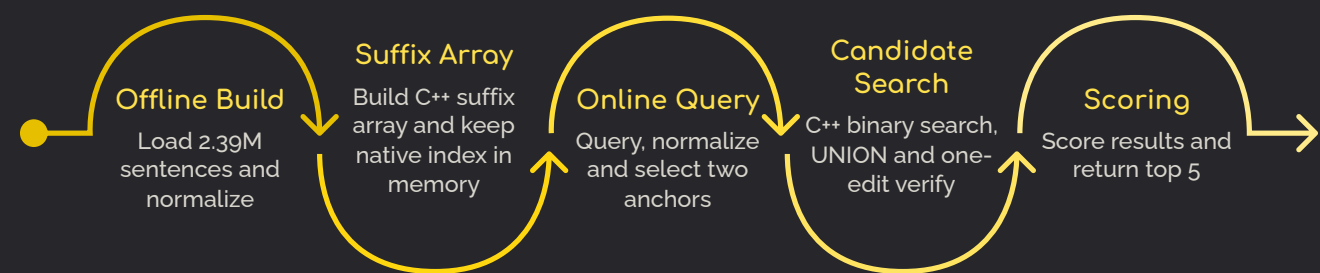
Build: 14.193 s · 211.5
MiB

C++ · 50K

Build: 0.579 s · 21.9
MiB

📄 **~24× faster build.** Moved only the measured hotspot to C++.

Build Once. Search Many Times.



Static archive → accept one-time build cost for fast repeated searches.

Full-Dataset Results

83s

Suffix Array Build

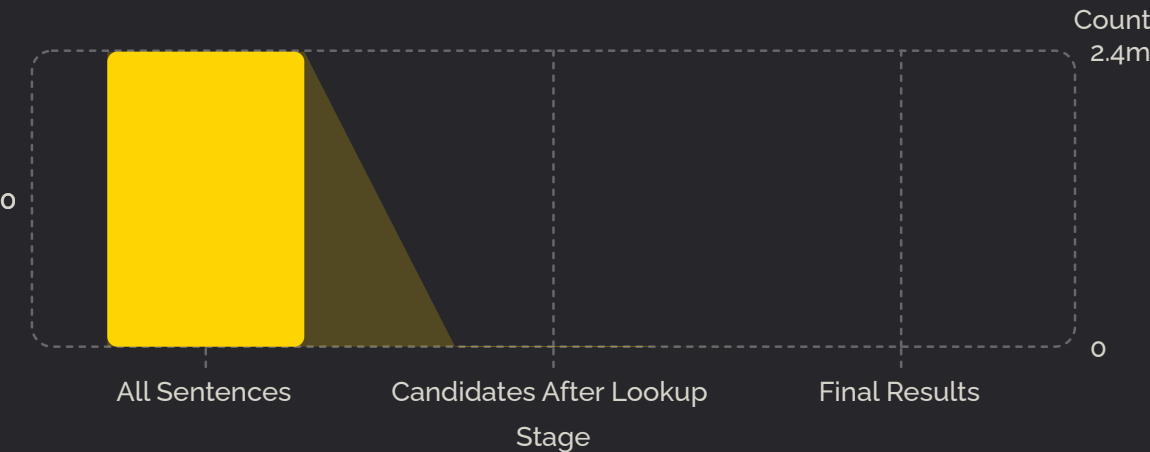
385MiB

Native Index

6ms

Search: "machine learning"

0%
Total conversion rate



Candidate Reduction

≈99.9% reduction · Longer query: 56 candidates, 0.376 ms

What We Learned

① Benchmark > Assumption

Build, test, measure — then decide.

② Prototype for Correctness, Optimize for Scale

Python for exploration. C++ for the hotspot.

③ Architecture Depends on the Workload

Static archive → build once. Dynamic data → different trade-off.

⚠ **Remaining Challenge — Short Queries:** "hi" → 38.8 s.
Next: optimize short-query retrieval without sacrificing correctness.

